

05주차 · 문장·문서 임베딩과 의미 유사도

- 연계 슬라이드: [05주차 슬라이드](#)
- 실습 Notebook: [week05.ipynb](#)
- 교재 연계: *Hands-On Large Language Models* 2장 일부와 10장
- 권장 학습 순서: 풀링 손계산 → 모델 구조 비교 → 손실함수 계산 → Notebook 검색 실험

1. 문장 임베딩이 해결하려는 문제

문장은 길이가 서로 다르다. 검색·군집화·중복 탐지에서는 수백만 문장을 빠르게 비교해야 하므로, 모든 문장 쌍을 Transformer에 함께 넣는 방식은 비용이 크다. 문장 임베딩(sentence embedding)은 가변 길이 텍스트 x 를 고정 길이 벡터로 바꾸는 함수다.

$$f_{\theta} : x \mapsto \mathbf{z} \in \mathbb{R}^d \tag{1}$$

“고정 길이 벡터”라는 형식만으로 좋은 문장 임베딩이 되는 것은 아니다. 과업에서 관련된 문장은 가깝고, 관련 없는 문장은 멀어지도록 학습목표와 데이터가 정렬되어야 한다. 검색에서는 문서 벡터를 미리 계산·색인할 수 있어야 하고, 새 질의도 같은 공간에서 비교할 수 있어야 한다.

좋은 문장·문서 임베딩의 조건은 다음과 같이 정리할 수 있다.

- 의미 또는 과업 관련성이 벡터 유사도에 반영된다.
- 새 입력을 다른 입력과 독립적으로 인코딩할 수 있다.
- 문서 벡터를 미리 계산해 근사 최근접 이웃 탐색에 사용할 수 있다.
- 언어, 길이, 도메인이 달라져도 평가 가능한 방식으로 일반화한다.
- 저장공간·처리량·지연시간이 서비스 요구를 만족한다.

2. 학습목표

학습을 마치면 다음을 할 수 있어야 한다.

- 토큰 은닉 상태를 문장 벡터로 합치는 풀링을 계산한다.

- 채움 토큰을 평균에 포함했을 때 생기는 편향을 수치로 설명한다.
- 교차 인코더, 이중 인코더, 후기 상호작용 모델을 비용과 정확도로 비교한다.
- 쌍, 삼중항, 배치 단위 학습 데이터의 관련·비관련 예시를 설계한다.
- 코사인 회귀, 삼중항 손실, InfoNCE/MNRL의 목적을 설명한다.
- 온도와 혼동 비관련 예시가 학습 신호에 미치는 영향을 계산한다.
- STS 평가와 검색 평가가 서로 다른 질문에 답한다는 점을 설명한다.
- Notebook에서 희소 기준선과 선택적 문장 임베딩 모델을 공정하게 비교한다.

3. 토큰에서 문장으로: 벡터 집계(풀링)

마지막 층의 토큰 은닉 상태를 $H = [\mathbf{h}_1, \dots, \mathbf{h}_T] \in \mathbb{R}^{T \times d}$ 라 하자. 이를 한 문장 벡터로 합치는 연산이 벡터 집계(풀링, pooling)다.

3.1 [CLS] 풀링

$$\mathbf{z} = \mathbf{h}_{\text{CLS}} \quad (2)$$

특수 토큰 [CLS]의 은닉 상태를 사용한다. 이 위치가 문장 전체를 대표하도록 학습되었는지가 중요하다. BERT 사전학습 출력의 [CLS]를 그대로 사용한다고 의미 유사도 공간이 보장되지는 않는다.

3.2 마스크 평균 풀링

유효 토큰 표시 $m_i \in \{0, 1\}$ 를 사용하면

$$\mathbf{z} = \frac{\sum_{i=1}^T m_i \mathbf{h}_i}{\sum_{i=1}^T m_i} \quad (3)$$

이다. 모든 유효 토큰의 정보를 사용하며 강한 기준선이지만, 특수 토큰 포함 여부와 긴 문장의 정보 희석을 확인해야 한다.

3.3 최댓값·가중 풀링

최댓값 풀링은 각 차원에서 가장 큰 값을 선택한다. 두드러진 특징을 보존하지만 작은 입력 변화에 민감할 수 있다. 가중 풀링은 학습된 가중치, 어텐션, IDF 등을 사용해 중요한 토큰을 더 크게 반영한다. 가중치를 사용했다고 곧바로 설명 가능성이 생기는 것은 아니다.

4. Notebook의 풀링 수치 예제

[week05.ipynb](#)은 다음 토큰 상태와 마스크를 사용한다.

$$H = \begin{bmatrix} 1 & 0 \\ 0.8 & 0.2 \\ 0 & 0 \end{bmatrix}, \quad m = [1, 1, 0] \tag{4}$$

세 행을 무조건 평균하면

$$\mathbf{z}_{\text{plain}} = [0.6, 0.0667] \tag{5}$$

이고, 실제 두 토큰만 평균하면

$$\mathbf{z}_{\text{masked}} = [0.9, 0.1] \tag{6}$$

이다. 이 예제에서는 채움 벡터가 0이어도 문장 벡터의 크기가 문장 길이에 따라 줄어든다. 채움 벡터가 0이 아닌 모델이라면 방향까지 달라질 수 있다. 코사인 유사도 전에 정규화하더라도 잘못 섞인 방향은 되돌릴 수 없으므로 마스크를 먼저 적용해야 한다.

Notebook 점검 1

1. 셋째 벡터를 $[5, -3]$ 으로 바꾸고 두 평균을 비교한다.
2. 유효 토큰 수가 0일 때 분모를 어떻게 안전하게 처리할지 설명한다.
3. 모델 카드에서 권장하는 풀링과 정규화 방식을 찾고 실험 기록에 남긴다.

5. 세 가지 문장 쌍 점수화 구조

5.1 교차 인코더

교차 인코더(cross-encoder)는 질의 q 와 문서 d 를 $[q; d]$ 처럼 한 입력으로 넣어 관련성 점수를 계산한다.

$$s(q, d) = g_\phi([q; d]) \quad (7)$$

모든 토큰이 서로 어텐션할 수 있어 세밀한 일치·부정·수치 관계를 포착하기 쉽다. 그러나 질의가 바뀔 때마다 모든 질의-문서 쌍을 다시 실행해야 한다. 문서가 N 개이고 질의가 Q 개라면 대략 QN 회의 모델 추론이 필요해 대규모 1차 검색에는 비싸다.

5.2 이중 인코더

이중 인코더(bi-encoder)는 질의와 문서를 독립적으로 인코딩한다.

$$\mathbf{z}_q = f_\theta(q), \quad \mathbf{z}_d = g_\theta(d), \quad s(q, d) = \cos(\mathbf{z}_q, \mathbf{z}_d) \quad (8)$$

문서 벡터를 한 번 계산해 색인할 수 있으므로 새 질의에는 질의 인코딩 한 번과 벡터 검색만 필요하다. 대신 각 텍스트를 한 벡터로 압축하므로, 질의-문서 토큰 사이의 세밀한 상호작용 일부를 잃을 수 있다.

5.3 후기 상호작용

후기 상호작용(late interaction)은 질의와 문서를 독립적으로 토큰 벡터 집합으로 인코딩한 뒤 검색 단계에서 토큰별 최대 유사도 등을 합친다. ColBERT의 대표 점수는 다음 형태다.

$$s(q, d) = \sum_{i \in q} \max_{j \in d} \mathbf{q}_i^\top \mathbf{d}_j \quad (9)$$

한 벡터보다 세밀하고 교차 인코더보다 재사용 가능하지만, 문서마다 여러 벡터를 저장하므로 색인 크기와 검색 비용이 증가한다.

6. SBERT가 바꾼 계산 구조

문장 n 개의 모든 쌍은 $n(n-1)/2$ 개다. $n = 10,000$ 이면 49,995,000쌍이다.

SBERT 논문은 당시 하드웨어에서 BERT 교차 인코더로 이 쌍들을 비교할 때 약 65시간이 들지만, 문장을 독립적으로 인코딩한 뒤 코사인 유사도를 계산하면 수 초 수준이라고 보고했다.

중요한 것은 오래된 절대시간이 아니라 복잡도의 차이다. 교차 인코더는 **쌍마다** 모델을 실행하고, 이중 인코더는 **문장마다** 한 번 실행한 뒤 저렴한 벡터 연산으로 재사용한다.

7. 논문 도판 읽기: Siamese SBERT

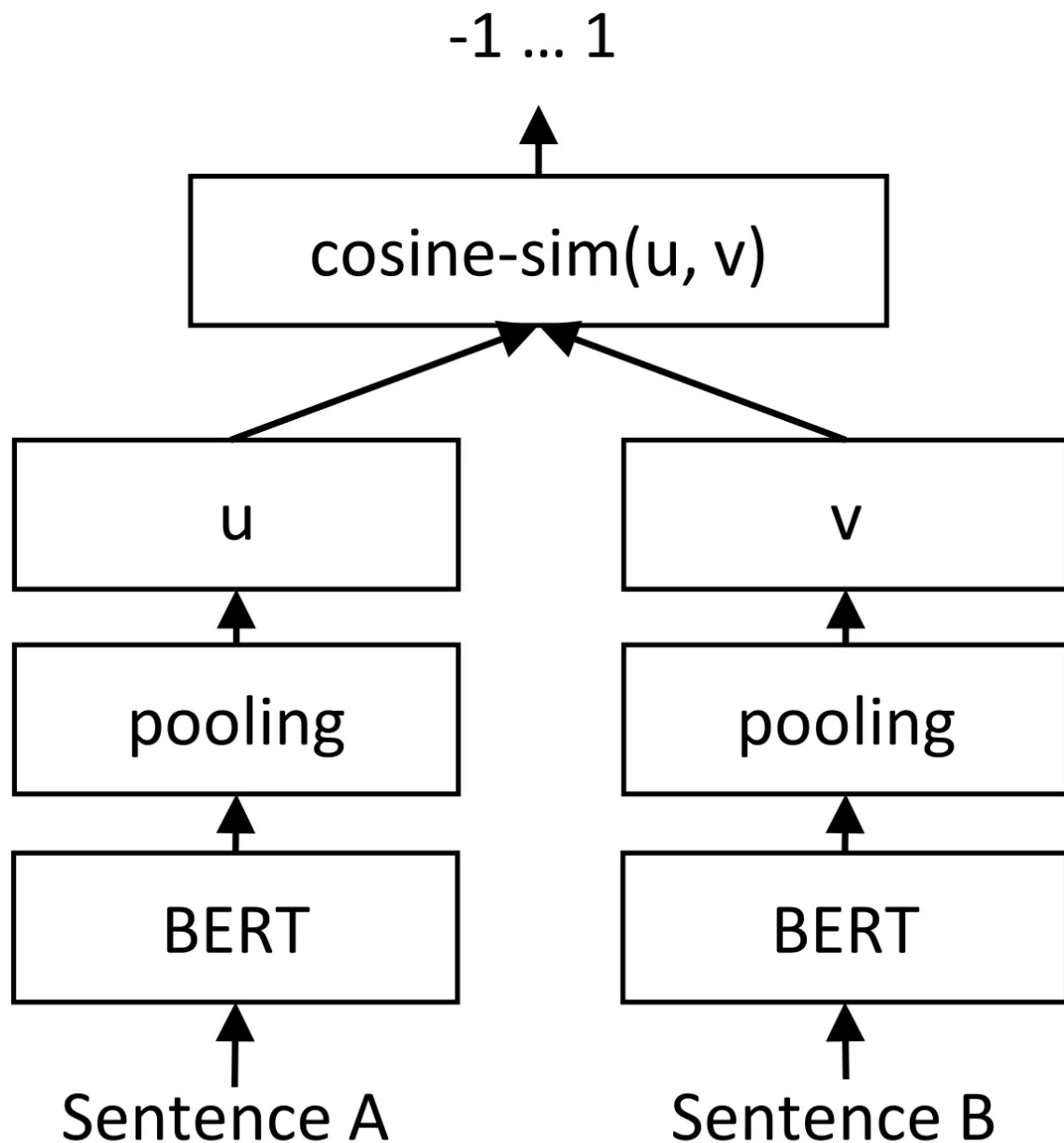


그림 1. Reimers & Gurevych (2019), "Sentence-BERT," Figure 2. 두 문장을 공유 가중치 BERT와 풀링으로 각각 벡터화한 뒤 코사인 유사도를 계산한다.

[원문과 Figure 2](#)

그림은 아래에서 위로 읽는다.

1. Sentence A와 Sentence B가 서로 다른 입력으로 들어간다.
2. 두 BERT 상자는 가중치를 공유하는 같은 인코더다. 이를 쌍둥이 신경망(siamese network) 구조라 한다.
3. 각 토큰 열을 풀링해 고정 길이 벡터 u, v 를 만든다.

4. 코사인 유사도로 두 벡터를 비교해 $[-1, 1]$ 범위 점수를 얻는다.

두 문장이 어텐션 단계에서 직접 만나지 않는다는 점이 핵심이다. 이 제약 덕분에 각 벡터를 재사용할 수 있지만, 두 문장 사이의 세밀한 상호작용은 학습을 통해 벡터 공간에 미리 압축되어야 한다.

8. 학습 데이터는 관련성의 정의다

관련 예시(positive)와 비관련 예시(negative)는 문장에 본래 붙어 있는 절대 속성이 아니다. 과업이 정의한 관계다. “청년 창업 지원”과 “청년 주거비 지원”은 주제 분류에서는 둘 다 청년 정책이라 관련될 수 있지만, 특정 창업 비용 검색에서는 서로 다른 답이다.

대표 데이터 구조는 다음과 같다.

- 쌍: (x_1, x_2, y) — 두 문장과 유사도 점수 또는 NLI 범주
- 삼중항: (a, p, n) — 기준 예시, 관련 예시, 비관련 예시
- 배치: $(q_i, d_i^+)_{i=1}^B$ — 다른 쌍의 문서를 배치 내 비관련 예시로 사용

분할할 때 같은 원문의 문장 변형이나 같은 문서의 청크가 학습·최종 평가에 나뉘어 들어가지 않도록 문서·주제·시간 단위 그룹 분할을 고려해야 한다.

9. 코사인 회귀와 분류형 쌍 학습

의미적 텍스트 유사도(semantic textual similarity, STS)의 정답 점수 $y \in [0, 1]$ 에 맞추려면 다음 평균제곱오차를 사용할 수 있다.

$$\mathcal{L}_{\text{MSE}} = (\cos(f(x_1), f(x_2)) - y)^2 \tag{10}$$

예측 코사인 유사도가 0.7이고 정답이 0.9라면 손실은 $(0.7 - 0.9)^2 = 0.04$ 다. 이 학습은 문장 쌍의 연속적 유사도를 맞추지만, 상위 문서 순위를 직접 최적화하는 것은 아니다.

NLI처럼 함의·중립·모순 범주가 있다면 $[u; v; |u - v|]$ 를 분류기에 넣을 수 있다. 다만 NLI의 “함의”와 검색의 “답을 포함한 문서”는 다른 관계이므로 배포 과업과의 전이 격차를 평가해야 한다.

10. 삼중항 손실의 손계산

삼중항 손실(triplet loss)은 관련 예시가 비관련 예시보다 여유값 m 만큼 가깝도록 요구한다.

$$\mathcal{L} = \max(d(a, p) - d(a, n) + m, 0) \quad (11)$$

$d(a, p) = 0.4, d(a, n) = 0.7, m = 0.5$ 라면

$$\mathcal{L} = \max(0.4 - 0.7 + 0.5, 0) = 0.2 \quad (12)$$

이다. 현재 거리 차이는 0.3인데 목표 여유는 0.5이므로 아직 0.2만큼 위반했다. 비관련 예시가 아주 멀리 있어 손실이 0이면 학습 신호가 없다. 반대로 실제로 관련된 문장을 잘못 비관련 예시로 표시하면 모델을 잘못된 방향으로 민다.

11. InfoNCE와 MNRL

배치의 (a_i, p_i) 만 정답 쌍이고 다른 p_j 를 비관련 예시로 간주하면

$$\mathcal{L}_i = -\log \frac{\exp(s(a_i, p_i)/\tau)}{\sum_{j=1}^B \exp(s(a_i, p_j)/\tau)} \quad (13)$$

와 같은 대조학습 손실을 쓸 수 있다. Sentence Transformers에서는 이 구조를 Multiple Negatives Ranking Loss(MNRL)로 널리 사용한다. 배치 크기 B 로 B^2 개의 점수를 만들기 때문에 효율적이다.

온도 τ 는 점수 차이를 확대하거나 완화한다. 관련 점수가 0.8, 두 비관련 점수가 0.4와 0.1일 때 $\tau = 0.1$ 이면 관련 예시 확률은 약 0.981이고 손실은 약 0.019다. $\tau = 1$ 이면 확률은 약 0.461, 손실은 약 0.773이다. 작은 온도는 작은 점수 차이를 크게 확대하지만, 잘못된 라벨에도 민감해질 수 있다.

큰 배치는 더 많은 비관련 예시를 제공하지만 중복 의미나 복수 정답이 있을 때 잘못된 비관련 판정이 늘어난다. 다중 관련 예시 손실, 중복 제거, 정답 목록 보완이 필요할 수 있다.

12. 혼동 비관련 예시

혼동 비관련 예시(hard negative)는 현재 모델이 높은 점수를 주지만 라벨 기준으로는 관련 없는 후보다. 쉬운 비관련 예시보다 결정 경계를 정교하게 만들 수 있다.

일반적인 절차는 다음과 같다.

1. 기준선 이중 인코더로 질의마다 상위 k 개를 검색한다.
2. 알려진 정답을 제외한 상위 후보를 비관련 후보로 모은다.

3. 사람 또는 교차 인코더로 실제 관련 문서가 섞였는지 다시 확인한다.
4. 쉬운 비관련 예시와 혼동 비관련 예시의 비율을 정해 재학습한다.
5. 별도의 개발 데이터에서 검색 지표와 오류 유형을 비교한다.

정답 목록이 불완전하면 “상위에 있지만 라벨이 없음”은 실제 비관련이 아닐 수 있다. 이 거짓 비관련 예시(false negative)를 그대로 학습하면 유용한 문서를 밀어내므로 가장 주의해야 한다.

13. 비대칭 검색과 지시문

STS는 비슷한 역할의 문장 두 개를 비교하지만, 검색의 질의와 문서는 역할이 다르다. 질의는 정보 요구이고 문서는 답의 근거다. 현대 임베딩 모델은 다음처럼 역할 접두문이나 지시문(instruction)을 요구할 수 있다.

```
1 query: 장학금 신청 마감일은?  
2 passage: 2026학년도 2학기 장학금 신청은 8월 21일까지입니다.
```

접두문은 코드 식별자 또는 모델이 학습한 문자열이므로 모델 카드의 정확한 형식을 따라야 한다. 접두문을 빠뜨리거나 질의와 문서에 뒤바뀌 붙이면 같은 모델이라도 검색 성능이 달라질 수 있다.

14. Hands-On LLM 10장의 미세 조정 읽기

교재의 코드를 다음 다섯 요소로 분해해 읽는다.

데이터 → 기반 모델과 풀링 → 손실함수 → 평가기 → 체크포인트 선택

반드시 기록할 항목은 모델과 데이터 버전, 입력 접두문, 최대 길이와 잘림, 정규화, 배치 크기, 학습률, 에포크, 난수 시드, 개발 데이터 선택 지표다. 최종 평가 데이터 점수를 보면서 손실함수나 체크포인트를 고르면 평가 누수다.

코사인 손실, MNRL, AugSBERT, TSDAE는 단순한 코드 조리법이 아니다. 각각 어떤 라벨 구조와 비관련 예시, 교사 모델, 비지도 복원 신호를 사용하는지 비교해야 한다.

`SentenceTransformer.encode` 를 읽을 때도 설정의 의미를 확인한다. `batch_size` 는 주로 처리량과 장치 메모리에, `normalize_embeddings` 는 내적의 해석에 영향을 준다. `model.max_seq_length` 보다 긴 입력은 잘릴 수 있으므로 손실 구간을 확인해야 한다. float16·int8 같은 낮은 정밀도 표현을 사용하면 저장공간은 줄지만 상위 문서 순위가 얼마나 바뀌는지 별도로 측정한다.

15. Notebook 검색 실험 읽기

Notebook에는 네 개 정책 문장과 “청년의 창업 비용을 도와주는 정책”이라는 질의가 있다.

항상 실행되는 기준선은 한국어 문자 2~4-그램 TF-IDF다. 선택 셀은 다국어 Sentence Transformers 모델로 밀집 임베딩을 만든다.

실행 전 가설

- 단어가 겹치는 “청년 창업 자금을 지원합니다”가 두 방법 모두에서 높을 것이다.
- “청년 사업가에게 사업비를 제공합니다”는 표면 단어 겹침이 적어 희소 기준선보다 밀집 모델이 상대적으로 높게 평가할 가능성이 있다.
- “고령자 이동 복지”와 “섬 지역 원격진료”는 비관련 예시여야 한다.

Notebook 점검 2

1. TF-IDF 결과의 전체 순위와 점수를 기록한다.
2. 인터넷이 가능한 환경에서 `RUN_SENTENCE_MODEL=True` 로 바꾸고 모델명을 기록한다.
3. `normalize_embeddings=True` 일 때 `E[1:] @ E[0]` 가 코사인 유사도인 이유를 설명한다.
4. 동의어를 쓴 문장, 같은 단어지만 의미가 다른 문장을 각각 두 개 추가한다.
5. 성공 사례와 실패 사례를 점수·순위·텍스트 근거로 한 개 이상 분석한다.

선택 모델을 실행하지 못해도 기준선, 풀링, 손실함수 손계산은 필수 학습 범위다. 저장된 임베딩을 사용한다면 모델명과 생성 조건을 함께 제공해야 재현할 수 있다.

16. 평가 과업을 구분하라

과업	입력과 정답	대표 지표	측정 질문
STS	문장 쌍 + 연속 점수	Spearman ρ	유사도 순서가 사람 점수와 맞는가?
분류	벡터 + 범주	정확도, 매크로 F1	범주 결정 경계를 만들 수 있는가?
군집화	벡터 집합	ARI, V-measure	구조가 외부 범주와 맞는가?
검색	질의 + 말뭉치 + 관련도	nDCG@k, Recall@k, MRR	정답 근거를 상위에 가져오는가?

STS에서 강한 모델이 장문 한국어 검색에서도 강하다는 보장은 없다. 언어, 길이, 도메인, 질의-문서 비대칭성, 정답 정의가 모두 다르기 때문이다. 실제 서비스와 가장 가까운 개발 데이터가 필요하다.

17. 최신 모델을 읽는 세 관점

BGE-M3는 한 모델에서 밀집, 희소, 다중 벡터 표현을 지원하는 방향을 제시했다.

[Gemini Embedding](#)은 대규모 다국어 임베딩 모델의 평가 범위를,

[jina-embeddings-v4](#)는 다중 모달·단일/다중 벡터와 과업별

어댑터의 결합을 탐구한다. Matryoshka Representation Learning은 앞부분 차원만 잘라도 성능을 보존하도록 학습해 저장공간과 검색 비용을 조절한다. Qwen3 Embedding 계열은 서로 다른 크기의 임베딩·재순위화 모델을 함께 제공한다. 모델 발표 점수는 동일한 데이터·과업·접근 조건에서 다시 검증해야 한다.

새 모델을 선택할 때는 다음 질문을 먼저 고정한다.

- 한국어와 해당 도메인 데이터에서 검증했는가?
- 최대 길이와 잘림 정책이 문서 단위에 맞는가?
- 한 벡터, 여러 벡터, 희소 표현 중 무엇을 저장하는가?
- 차원 축소 후 상위 k 개 재현율과 지연시간은 얼마나 변하는가?
- 라이선스, 모델 크기, 장치 메모리가 배포 조건과 맞는가?

예를 들어 1024차원 float32 벡터 100만 개의 원시 저장공간은 약 3.81GiB다. 256차원으로 줄이면 약 0.95GiB다. 그러나 4배 작아졌다는 사실만으로 채택하지 말고 자신의 말뭉치에서 검색 품질을 다시 측정해야 한다.

18. 자주 하는 오해

1. 평균 풀링은 채움 토큰이 0이면 안전하다. 유효 토큰 수가 달라져 크기 편향이 생긴다.
2. 교차 인코더가 더 정확하므로 모든 문서에 적용하면 된다. 대규모 1차 검색 비용을 감당하기 어렵다.
3. 이중 인코더의 두 상자는 서로 다른 모델이다. 보통 가중치를 공유하지만 비대칭 구조도 가능하다.
4. 코사인 유사도가 높으면 두 문장은 같은 뜻이다. 모델이 학습한 관련성에 따른 점수일 뿐이다.
5. 배치의 다른 문서는 모두 비관련이다. 복수 정답과 중복 의미 때문에 거짓 비관련 예시가 생길 수 있다.
6. STS 점수만 높으면 검색 모델로 충분하다. 검색에는 순위와 정답 근거 재현이 중요하다.

7. 차원이 크면 항상 더 좋다. 저장공간·지연시간과 실제 성능의 상충 관계를 측정해야 한다.

19. 핵심 정리

- 문장 임베딩은 텍스트를 비교 가능한 고정 길이 벡터로 바꾸는 과업 목적 표현이다.
- 풀링에서는 채움 토큰 마스크와 특수 토큰 처리 규칙이 중요하다.
- 교차 인코더는 세밀하지만 비싸고, 이중 인코더는 사전 계산으로 확장성을 얻는다.
- 후기 상호작용은 여러 토큰 벡터를 저장해 정확도와 비용 사이의 중간점을 만든다.
- 손실함수보다 먼저 관련·비관련 예시가 과업에서 무엇인지 정의해야 한다.
- 혼동 비관련 예시는 강한 신호지만 거짓 비관련 예시 검증이 필요하다.
- 모델 선택은 언어·도메인·길이·지표·비용을 고정한 실험으로 결정한다.

20. 복습 문제

1. 마스크 평균 풀링의 식을 쓰고 분모가 무엇을 뜻하는지 설명하시오.
2. 문서 100만 개에 새 질의 1,000개가 들어올 때 교차 인코더와 이중 인코더의 계산 차이를 설명하시오.
3. SBERT Figure 2에서 두 BERT가 가중치를 공유하는 이유는 무엇인가?
4. $d(a, p) = 0.2$, $d(a, n) = 0.9$, $m = 0.5$ 일 때 삼중항 손실을 계산하시오.
5. 온도 τ 가 작아질 때 대조학습 확률 분포가 어떻게 바뀌는가?
6. 배치 내 비관련 예시에서 거짓 비관련 판정이 발생하는 사례를 하나 만드시오.
7. 대칭형 STS와 비대칭 검색의 입력과 평가 지표를 비교하시오.
8. 모델 접두문을 요소 제거 실험에 포함해야 하는 이유를 설명하시오.
9. 1024차원과 256차원 모델을 비교하는 최소 실험표를 설계하시오.

참고문헌

1. Reimers, N. & Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." [arXiv](#)
2. Henderson, M. et al. (2017). "Efficient Natural Language Response Suggestion for Smart Reply."

[arXiv](#)

3. Karpukhin, V. et al. (2020). “Dense Passage Retrieval for Open-Domain Question Answering.” [EMNLP](#)
4. Khattab, O. & Zaharia, M. (2020). “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction.” [SIGIR](#)
5. Gao, T. et al. (2021). “SimCSE: Simple Contrastive Learning of Sentence Embeddings.” [EMNLP](#)
6. Wang, L. et al. (2022). “Text Embeddings by Weakly-Supervised Contrastive Pre-training.” [arXiv](#)
7. Kusupati, A. et al. (2022). “Matryoshka Representation Learning.” [NeurIPS](#)
8. Chen, J. et al. (2024). “BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings.” [ACL Anthology](#)
9. Zhang, Y. et al. (2025). “Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models.” [arXiv](#)
10. Hands-On LLM Authors. *Hands-On Large Language Models*, [Chapter 2](#) and [Chapter 10](#)